逆向工程 & 密码学

Date: September 2, 2026 上次更新于2026.08

Reverse & Crypto

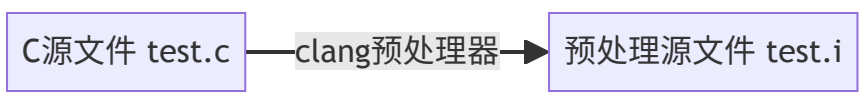
Press Space for next page →

二进制基础–从源文件到可执行程序



预处理阶段

1	预处理Preprocessing	展开所有的`#define`宏定义； 处理`#if`、`#ifdef`、`#elif`、`#else`、`#endif`等； 处理`#inlcude`，将被包含的文件直接插入； 删除注释； 添加行号和文件标识； 保留`#program`编译器指令； `gcc -E hello.c -o hello.i`进行预处理，输出文件.i	预处理器cpp

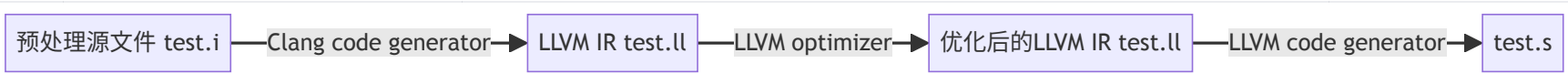


二进制基础–从源文件到可执行程序



编译阶段

2	编译compilation	词法分析，得到一系列token 语法分析，生成语法树 语义分析，静态语义 + 动态语义 源代码优化，将语法树转化为中间代码 目标代码生成 目标代码优化 输出文本文件.s		编译器ccl



二进制基础–从源文件到可执行程序



汇编阶段

3	汇编assembly	将.s文本文件翻译为机器语言指令 将指令打包为 `可重定位目标程序` 格式 输出二进制文件.o		汇编器as

目标文件 .o

可以使用IDA查看内部信息

FILE header 、 .text section 、 .data section 、 .bss section

内存结构

静态存储区、动态存储区（栈、堆）、代码区

高地址	内核	存储内容	备注
.text（只读）	代码段	可执行代码	有效防止堆栈溢出； 共享的；只读的； 经编译并链接后得到， 是程序代码在内存中的映射

关于 `.text` 段是**共享**的：

对于经常执行的程序，只需要存储一个副本在内存中。

关于 `.text` 段是**只读**的：

有效防止程序以外修改指令。

内存结构

静态存储区、动态存储区（栈、堆）、代码区

高地址	内核	存储内容	备注
.data	读写数据段.WRdata	初始化的全局变量和静态变量	由程序员初始化，经编译并链接后得到，静态内存分配
.data（只读）	只读数据段.ROdata	初始化的常变量，如字符串常量	由`const`修饰的全局变量，经编译并链接后得到，静态内存分配
.text（只读）			

内存结构

静态存储区、动态存储区（栈、堆）、代码区

高地址	内核	存储内容	备注
.bss		未初始化的全局变量和静态变量	由 `global` 或 `static` 修饰； 运行前由内核初始化为0，静态内存分配
.data			
.text			

> 关于 `.bss` 段的命名由来：
> IBM 704汇编语言“块存储开始block storage start”指令，
> 也可以记为“better save space”。

高地址	内核	存储内容	备注
⬇	栈	返回地址；函数参数；局部变量（非 `static` ）	运行时动态生成，即，动态内存分配
…	栈空间		由高向低， 快速，连续，空间有限
内存映射	高效的I/O映射	文件映射、动态库、匿名映射	用于加载一个共享的动态内存库。 用户可使用系统interface创建共享内存， 用作**进程通信** 共享库的内存映射，如printf()函数
…	堆空间	堆区域由所有共享库和进程中动态加载的模块共享	由低向高， 缓慢，不连续（链表），空间大
⬆	堆		**是由程序员自己分配的** 动态内存分配

？常见的寄存器有哪些

二进制基础–从源文件到可执行程序



链接阶段

4	链接linking	将程序中调用的诸如printf.o等预编译好的目标文件合并进程序 输出可执行目标文件	连接器linker

> linker
> 可以执行于编译时（源代码翻译为机器代码时），或者加载时（程序被加载到内存并执行时），甚至是运行时（应用程序来执行链接）
> 使得分离编译成为可能
> 分为静态链接、动态链接

链接阶段

静态链接、动态链接

- 静态链接
  - 将头文件的库全部加载到链接后的可执行文件中
- 动态链接
  - 将某些符号的重定位推迟到程序执行阶段
  - 减少库中无用函数带来的可执行文件过大

动态链接的存在合理性：
静态链接中，假设两个程序共享一个模块，则静态链接后输出的两个可执行文件中各有一个共享模块的副本，
若同时运行这两个可执行文件，
则**共享模块**在磁盘和内存中都有**两个副本**，造成了磁盘和内存的极大浪费。

动态链接

动态链接共享库shared library

> 共享库的存在合理性： 静态库需要定期的维护和更新

DLL和SO都是啥

> `DLL` : Dynamic link libraries
>
> `SO` : Shared object

- `dll` 和 `so` 是不同OS上的动态链接库
  - `dll` 是window使用的动态链接库格式，通常用于windows平台上的程序；Windows程序下用 `LoadLibrary` 和 `GetProAddress` 等程序动态加载
  - `so` 是类unix系统使用的动态链接库格式，通常用于类unix系统上的程序；类Unix程序使用 `dlopen` 和 `dlsym` 等函数动态加载

- **共享库.so**是一个目标模块
  - 在运行或加载时，可以加载到任意的内存位置，并和内存中的程序链接，整个过程称为 **动态链接**

ELF-64 可重定位目标文件

ELF(executable and linkable format)

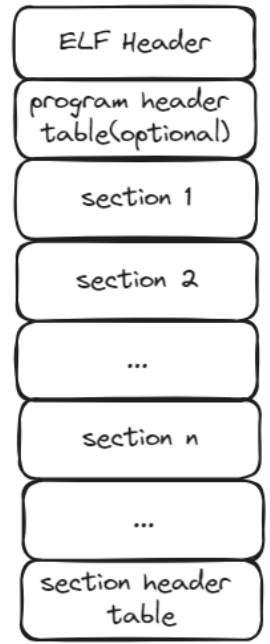
- 常见的文件类型包括：
  - 可执行文件、
  - 共享库.so、
  - 目标文件.o
  - core文件

ELF链接视图

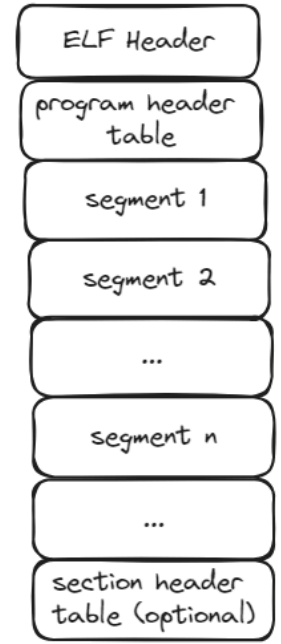
> ELF Linking View
> 链接阶段，可以忽略 `program header table`

ELF执行视图

> ELF Execution View
> 运行阶段，可以忽略 `section header table`



LINKING View



EXECUTION View

.got 全局偏移表

用于地址解析，RW数据段

- 当`hello`程序要访问`共享库中的全局变量`时，编译器和链接器会执行以下步骤

  `mov 0x200271(%rip),%rax # 200828 <_DYNAMIC+0x1a0>`

  `mov (%rax),%eax`

  - 第一条指令从`.got`表中加载变量的实际地址，动态链接器将其保存至`%rax`；
  - 第二条指令通过`%rax`里的值间接访问变量的实际地址

- GOT是**数据段**中确定全局变量和外部函数地址的表
  - 有两种GOT表：
    - `.got`表用于保存`全局变量`的引用地址
    - `.got.plt`表用于保存`函数`的引用地址

.plt 过程链接表

调用外部function或procedure，代码段

- 第一次调用外部函数 `call hello@plt` 时：
  - 此时 `hello@got.plt` 中没有 `hello` 的实际地址，而是指向 PLT 中用于触发动态解析的代码；
  - `hello@plt` 根据预先设置的重定位信息，将控制权转交给 PLT 的公共解析入口（`plt0`），进一步调用动态链接器的 resolver；
  - 动态链接器根据 `.rela.plt` 等重定位信息查找 `hello` 的实际地址；
  - 动态链接器将解析的实际地址写入 `hello@got.plt` 对应的 GOT 表项；随后跳到 `hello` 实际地址。
- 后续再次调用该函数时：
  - 程序仍然通过 `hello@plt` 进入 PLT；直接通过 `hello@got.plt` 进行间接跳转；

> 延迟绑定
> 动态链接中，函数在第一次被调用时，才进行符号查找、重定位等，如果函数不被调用就不进行绑定，以加快程序启动速度
> PLT是用于延迟绑定的的表

Reverse

常用的静态分析工具:

- IDA Pro
- Ghidra
  - ✏使用教程
- 示例: BombLab

pwndbg

GDB插件，调试辅助工具。主要功能:辅助调试和逆向

安装

```
git clone https://github.com/pwndbg/pwndbg
```

```
cd pwndbg
```

```
./setup.sh
```

pwntools

Python库，编写自动化利用脚本。

安装:

```
pip3 install pwntools
```

使用1：

```
from pwn import *
# 本地交互
io = process("./bomb")
# 远程交互
io = remote('exploitme.example.com', 3333)
# phase_1: 在收到 "Have a nice day!\n" 后发送 b"Border relations with Canada have never been better.\n"
io.sendlineafter(b"Have a nice day!\n", b"Border relations with Canada have never been better.")
# 日志打印: 提示信息
log.info("phase_1 done.")

# 进入交互式会话
io.interactive()
```

- 更多内容可查询文档 👉 https://docs.pwntools.com/en/stable/

pwntools

Python库，编写自动化利用脚本。

使用2：

```
#!/bin/python3
from pwn import *

# 载入elf文件信息，可获取符号表,代码段,plt,got等信息
elf = ELF('./pwnme')
log.info("base@ {:#x}".format(elf.address)) # 文件装载的基地址
log.info("stdout@ {:#x}".format(elf.sym.stdout)) # 符号表中 stdout 的偏移地址【这里 stdout 是全局变量】
log.info("main@ {:#x}".format(elf.sym.main)) # 符号表中 main 函数的偏移地址
log.info("printf[got]@ {:#x}".format(elf.got.printf)) # got 表中 printf 函数地址
log.info("printf[plt]@ {:#x}".format(elf.plt.printf)) # plt 表中 printf 函数地址
# 进入交互模式
io.interactive()
```

pwntools

Python库，编写自动化利用脚本。

使用总结

```
io.recv() # 不间断接收数据
io.recvline() # 接收一行数据，遇到换行符结束
io.recvuntil(b'123') # 接收到123则结束，可添加drop=True丢弃123
io.recvall() # 接受全部，遇到EOF结束
io.send(data) # 发送数据
io.sendline(data) # 发送一行数据
io.sendafter(delims, data) # 在接收到delims之后发送数据
```

```
# 将integer打包为bytes，即整数转换为字符串
# 可用于伪造返回地址或覆盖内存，进而构造payload
p32(0xdeadbeef)
p64(0xdeadbeef)
# 拆包
# 从内存读取数据时还原整数值
u32(b'\xef\xbe\xad\xde') # 小端存储
u64(b'\xef\xbe\xad\xde\x00\x00\x00\x00')
```

题目分析

Reverse

IDA反编译

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  init(argc, argv, envp);
  parse_post_data(&post_data, username, password);
  printf("[*] Login info: username=%s, password=%s<br>", username, password);
  phase_1(username, password);
  return 0;
}
```

```
int __fastcall phase_1(__int64 a1, __int64 a2)
{
  char *v3; // [rsp+18h] [rbp-8h]

  if ( !(unsigned int)check_username(a1) || !(unsigned int)check_password(a2) )
    return puts("Login failed!");
  printf("Login success, here is your flag: ");
  v3 = getenv("REV_FLAG1_PATH");
  printcontent(v3);
  return phase_2();
}
```

```
_BOOL8 __fastcall check_username(const char *a1)
{
  size_t v1; // rax

  v1 = strlen(login_name);
  return strncmp(a1, login_name, v1) == 0;
}
```

进一步去查看 `login_name` 的值... `password` 类似

得到username和password后post提交，并查看返回信息

```
[*] Login info: username=***, password=***
Login success, here is your flag: flag{***}
Here is another one question hint info: transpose
Here is another flag:
0x19,0xffffffa5,0xffffffc9,0xffffffe8,0xffffffa4,0xffffffb8,0x08,0xffffffda,0xffffffd0,0xffffff8a,0xffffffc9,0xffffffc5,0x6
```

继续回到IDA分析phase-2...

Reverse done.

Crypto

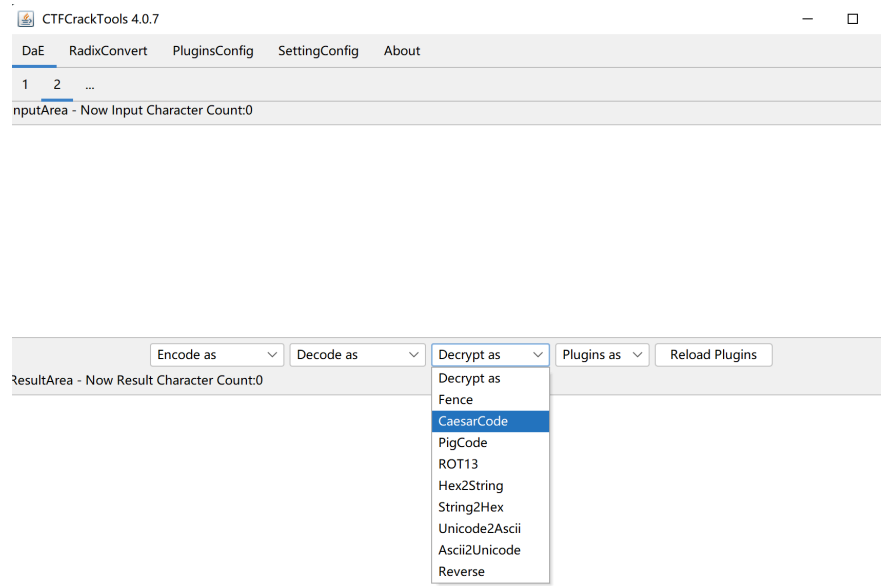
古典密码与现代密码

古典密码

- 📙 Wikipedia - 古典密码
  - 替换式密码
    - 凯撒密码
    - 维吉尼亚密码
      - 破解工具:

        https://www.guballa.de/vigenere-solver
    - ...
  - 移项式密码
    - 栅栏密码
    - ...

- CTFCrackTools



- CyberChef

现代密码

非对称加密算法 RSA

- 非对称加密算法指的是 加、解密使用不同的密钥，一把为公开的公钥，另一把为私钥。
- 公钥加密，私钥解密的过程，称为「加密」
- 私钥加密，公钥解密的过程，称为「签名」

对称加密算法 AES

AES加密/解密算法是一种可逆的对称加密算法，这类算法在加密和解密时使用相同的密钥，一般用于服务端对服务端之间对数据进行加密/解密。

题目分析

Crypto

Hint: Caesar Cipher
GET with `?data=` to submit correct data to pass the phase_1.
Here is the encrypted data:
PcnxP3jyOJUoOpZzOy1cDcDyBJG3D2OjOMLyOy0pDzG1EJLaPzVyCTL9Sw==

Crypto Done.

实验报告

- 命名方式：学号-姓名-RE.pdf
- 截止日期：夏季学期**第三周周五晚8点**前
- 要求：
  - **补充要求：内容精简，不要有废话、套话**